Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №3
дисциплины «Программирование на Python»

Выполнил:
Оразов Тимур
2 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

_____
(подпись)

Руководитель практики:
Воронкин Р. А.

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты_____

Ставрополь, 2025 г.

Вариант 16

Тема: Условные операторы и циклы в языке Python

Цель: приобретение навыков программирования разветвляющихся алгоритмов и алгоритмов циклической структуры. Освоить операторы языка Python версии 3.x if, while, for, break u continue, позволяющих реализовать разветвляющиеся алгоритмы и алгоритмы циклической структуры.

Ссылка на репозиторий с которого выполнялась работа:
https://github.com/Tim120903/laba_3

Ход работы:

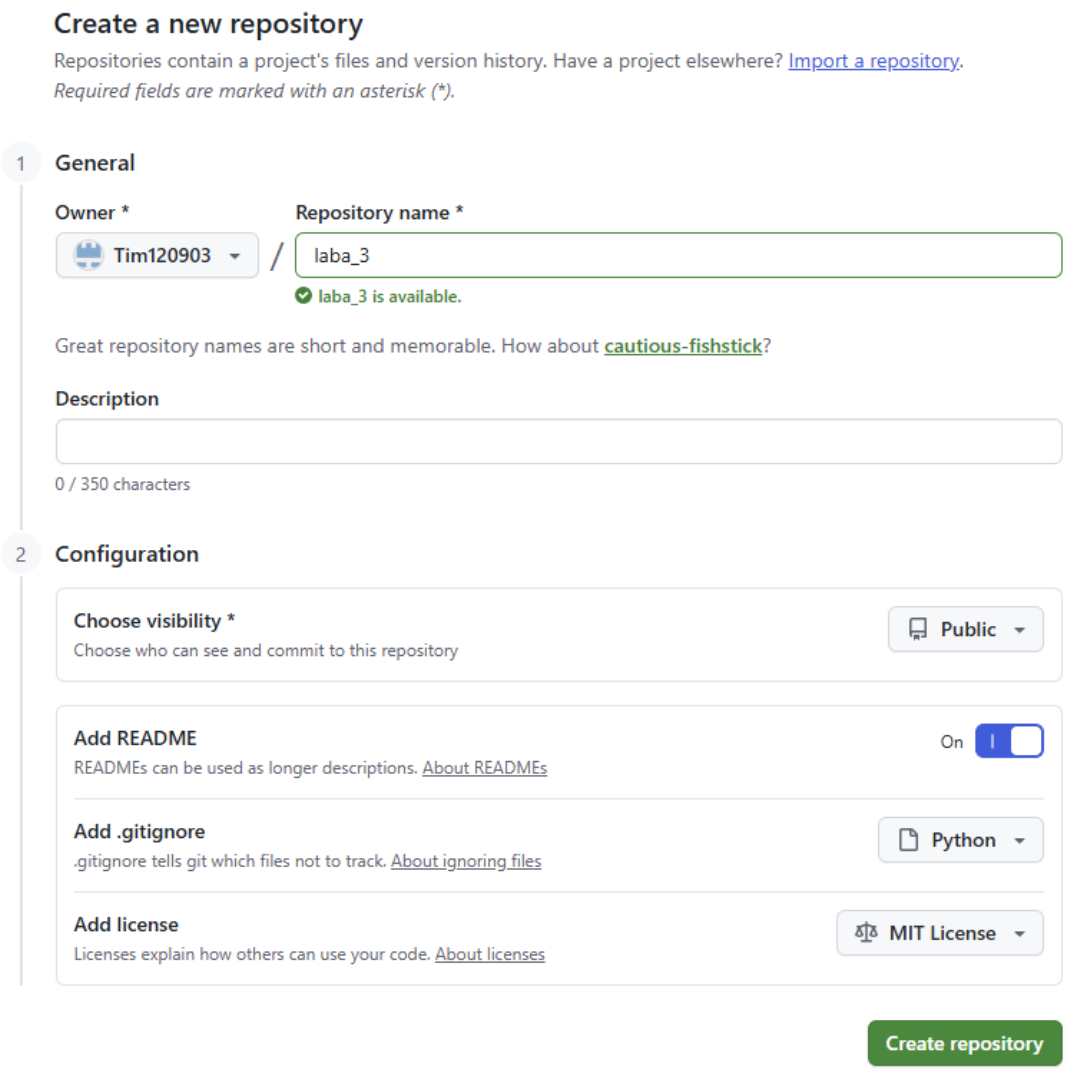

Рис. 1- создание общедоступного репозитория.

```
C:\Users\Тимур>git clone https://github.com/Tim120903/laba_3.git
Cloning into 'laba_3'
```

Рис. 2 - копирование репозитория на устройство.

```
# Covers JetBrains IDEs: IntelliJ, GoLand, RubyMine, PhpStorm, AppCode, PyCharm
# Reference: https://intellij-support.jetbrains.com/hc/en-us/articles/206544839

# User-specific stuff
.idea/**/workspace.xml
.idea/**/tasks.xml
.idea/**/usage.statistics.xml
.idea/**/dictionaries
.idea/**/shelf

# AWS User-specific
.idea/**/aws.xml

# Generated files
.idea/**/contentModel.xml

# Sensitive or high-churn files
.idea/**/dataSources/
.idea/**/dataSources.ids
.idea/**/dataSources.local.xml
.idea/**/sqlDataSources.xml
.idea/**/dynamic.xml
.idea/**/uiDesigner.xml
.idea/**/dbnavigator.xml

# Gradle
.idea/**/gradle.xml
.idea/**/libraries

# Gradle and Maven with auto-import
# When using Gradle or Maven with auto-import, you should exclude module files,
# since they will be recreated, and may cause churn.  Uncomment if using
# auto-import.
```

Рис. 3 - Добавленные правила в файл .gitignore

primer_1.py

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import math


if __name__ == '__main__':
    x = float(input("Value of x? "))

    if x <= 0:
        y = 2 * x * x + math.cos(x)
    elif x < 5:
        y = x + 1
    else:
        y = math.sin(x) - x * x

    print(f"y = {y}")
```

Run primer_1

```
C:\Users\Тимур\laba_3\.venv\Scripts\python.exe C:\Users\Тимур\laba_3\primer_1.py
Value of x? 324
y = -104976.40406521945

Process finished with exit code 0
```

Рис. 4 -Пример 1.

primer_1.py Primer_2.py

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import sys


if __name__ == '__main__':
    n = int(input("Enter the month number:"))

    if n == 1 or n == 2 or n == 12:
        print("Winter")
    elif n == 3 or n == 4 or n == 5:
        print("Spring")
    elif n == 6 or n == 7 or n == 8:
        print("Summer")
    elif n == 9 or n == 10 or n == 11:
        print("Autumn")
    else:
        print("Error!", file=sys.stderr)
        exit(1)
```

Run Primer_2

```
C:\Users\Тимур\laba_3\.venv\Scripts\python.exe C:\Users\Тимур\laba_3\Primer_2.py
Enter the month number: 4
Spring

Process finished with exit code 0
```

Рис. 5 - Пример 2.

primer_1.py Primer_2.py Primer_3.py

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import math


if __name__ == '__main__':
    n = int(input("Value of n? "))
    x = float(input("Value of x? "))

    S = 0.0
    for k in range(1, n+1):
        a = math.log(k * x) / (k * k)
        S += a

    print(f"S = {S}")
```

Run Primer_3

```
C:\Users\Тимур\laba_3\.venv\Scripts\python.exe C:\Users\Тимур\laba_3\Primer_3.py
Value of n? 456
Value of x? 1.14654
S = 1.146586519919472

Process finished with exit code 0
```

Рис. 6 - Пример 3.

primer_1.py × Primer_2.py Primer_3.py Primer_4.py ×

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import math
import sys


if __name__ == '__main__':
    a = float(input("Value of a? "))
    if a < 0:
        print("Illegal value of a", file=sys.stderr)
        exit(1)

    x, eps = 1, 1e-10
    while True:
        xp = x
        x = (x + a / x) / 2
        if math.fabs(x - xp) < eps:
            break

    print(f"x = {x}\nX = {math.sqrt(a)}")
```

Run Primer_4 ×

```
C:\Users\Тимур\laba_3\.venv\Scripts\python.exe C:\Users\Тимур\laba_3\Primer_4.py
Value of a? 5.645
x = 2.375920874103344
X = 2.375920874103344

Process finished with exit code 0
```

Рис. 7 - Пример 4.

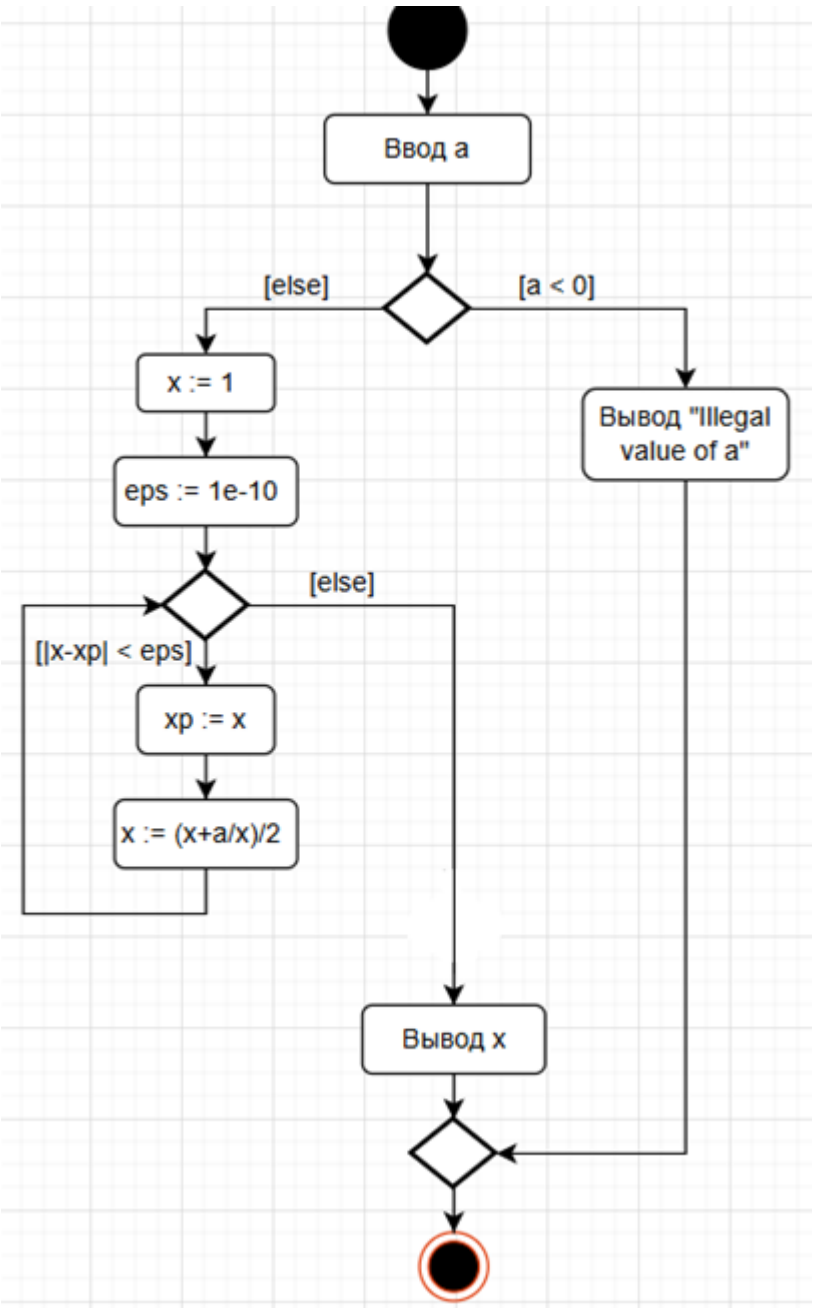

Рис. 8 – UML-диаграмма к примеру 4.

primer_1.py × Primer_2.py Primer_3.py Primer_4.py Primer_5.py ×

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import math
import sys


EULER = 0.5772156649015328606
EPS = 1e-10


if __name__ == '__main__':
    x = float(input("Value of x: "))
    if x == 0:
        print("Illegal value of x", file=sys.stderr)
        exit(1)

    a = x
    S, k = a, 1

    while math.fabs(a) > EPS:
        a *= x * k / (k + 1) ** 2
        S += a
        k += 1

    print(f"Ei({x}) = {EULER + math.log(math.fabs(x)) + S}")
```

Run Primer_5 ×

```
C:\Users\Тимур\laba_3\.venv\Scripts\python.exe C:\Users\Тимур\laba_3\Primer_5.py
Value of x: 43
Ei(43.0) = 1.1263482901669667e+17

Process finished with exit code 0
```

Рис. 9 – Пример 5.

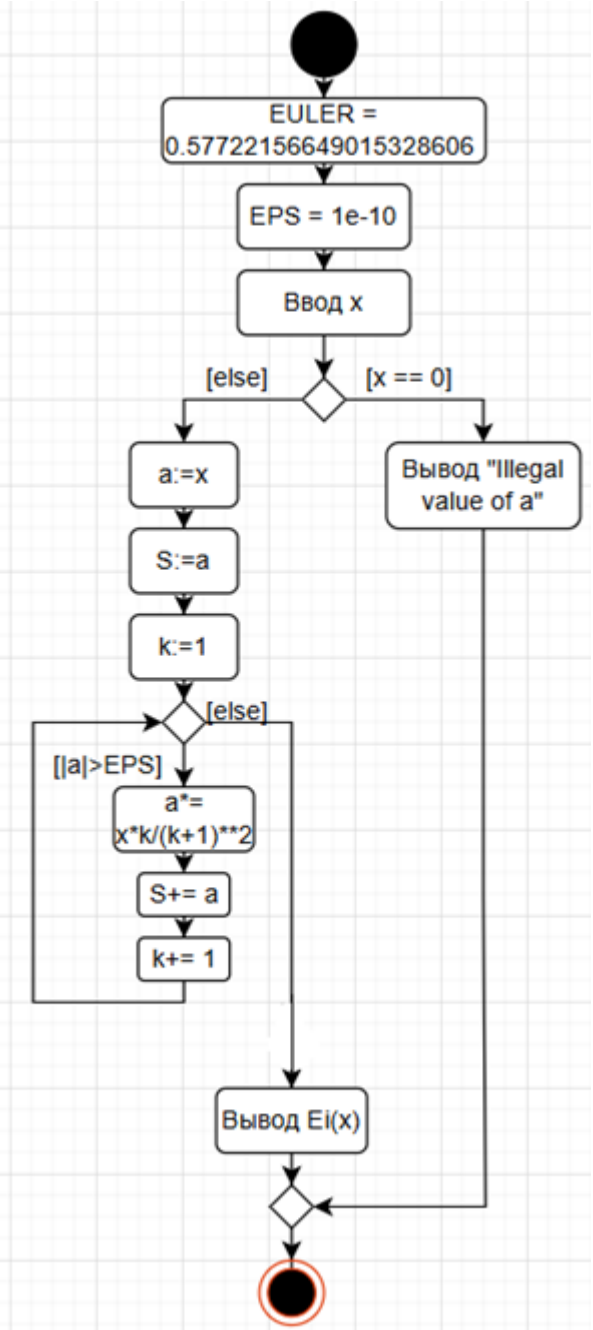

Рис. 10 – UML-диаграмма к примеру 5.

3. Дано число m ($1 \leq m \leq 7$). Вывести на экран название дня недели, который соответствует этому номеру.

Рис. 11 – индивидуальное задание 1 (В-16).

primer_1.py | Primer_2.py | Primer_3.py | Primer_4.py | Primer_5.py | individualnoe zadanie_1.py

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import sys


if __name__ == '__main__':
    a = int(input("Value a from 1 to 7: "))

    if a == 1:
        print("Monday")
    elif a == 2:
        print("Tuesday")
    elif a == 3:
        print("Wednesday")
    elif a == 4:
        print("Thursday")
    elif a == 5:
        print("Friday")
    elif a == 6:
        print("Saturday")
    elif a == 7:
        print("Sunday")
    else:
        print("Illegal value of a", file=sys.stderr)
        exit(1)
```

Run | individualnoe zadanie_1

```
C:\Users\Тимур\laba_3\.venv\Scripts\python.exe "C:\Users\Тимур\laba_3\individualnoe zadanie_1.py"
Value a from 1 to 7: 2
Tuesday

Process finished with exit code 0
```

Рис. 12- результат выполнения индивидуального задания 1.

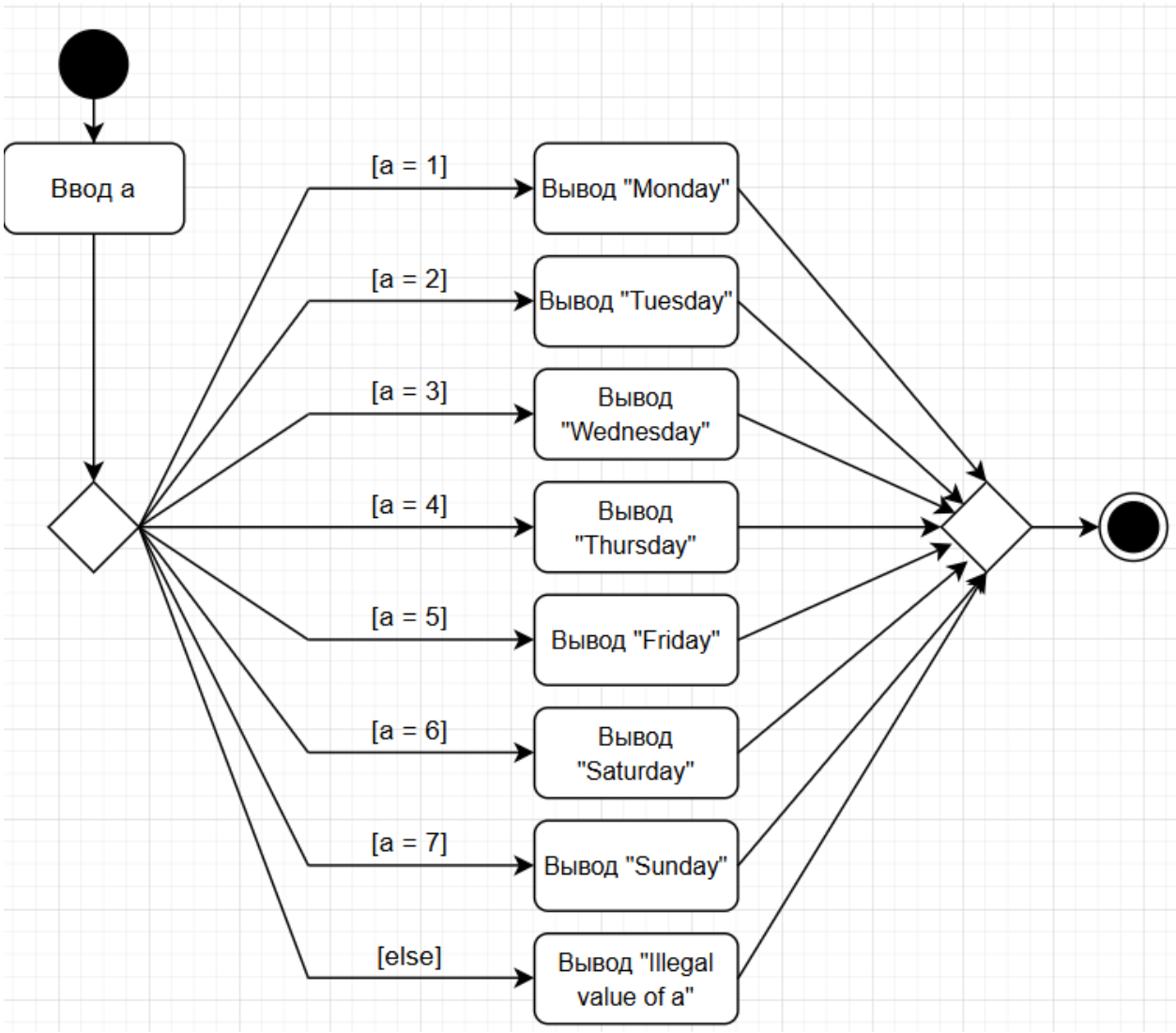

Рис. 13- UML-диаграмма к индивидуальному заданию 1.

16. Даны три действительных числа. Составить программу, выбирающую из них те, которые принадлежат интервалу $(0, 1)$.

Рис. 14 – индивидуальное задание 2 (В-16).

Primer_5.py × individualnoe zadanie_2.py ×

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import sys


if __name__ == '__main__':
    a = float(input("Value a from 0 to 1: "))
    b = float(input("Value a from 0 to 1: "))
    c = float(input("Value a from 0 to 1: "))

    if 0 < a < 1:
        print("The correct value", a)
    else:
        print("Wrong value", a)


    if 0 < b < 1:
        print("The correct value", b)
    else:
        print("Wrong value", b)

    if 0 < c < 1:
        print("The correct value", c)
    else:
        print("Wrong value", c)
```

Run individualnoe zadanie_2 ×

```
Value a from 0 to 1: 1
Value a from 0 to 1: 0.94848
Value a from 0 to 1: 5
Wrong value 1.0
The correct value 0.94848
Wrong value 5.0

Process finished with exit code 0
```

Рис. 15- результат выполнения индивидуального задания 2.

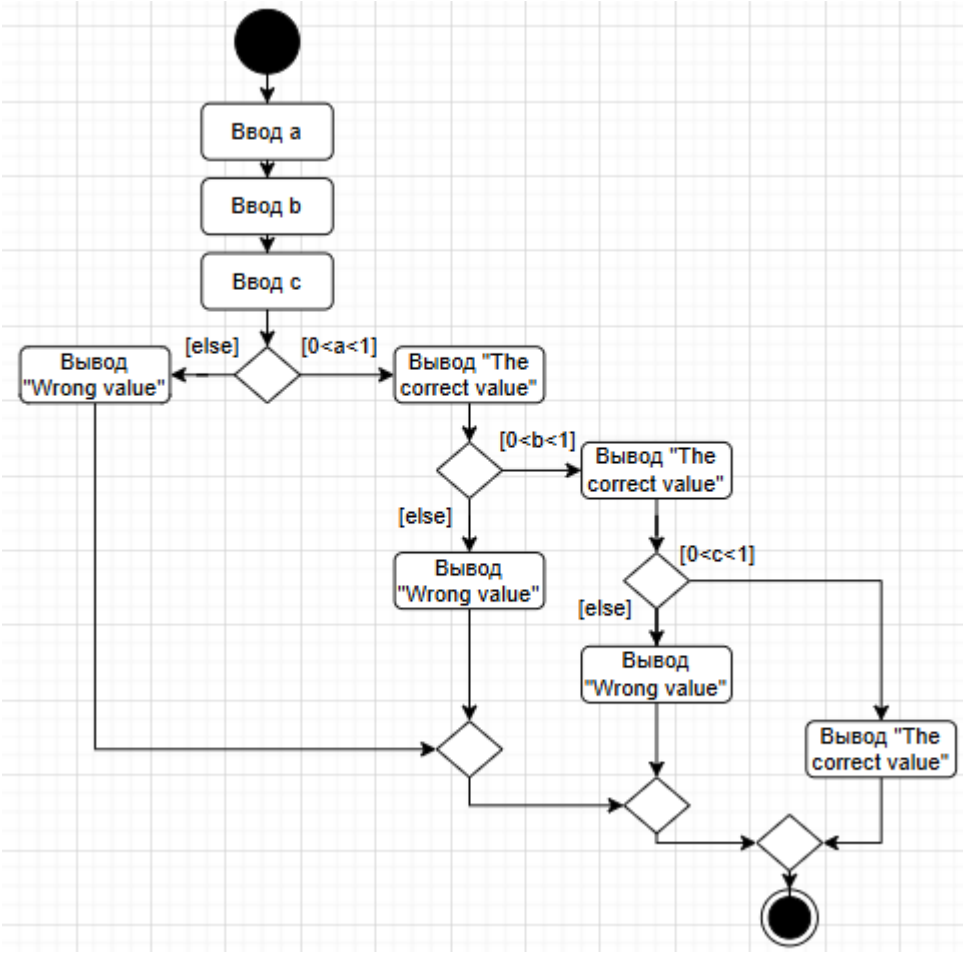

Рис. 16- UML-диаграмма к индивидуальному заданию 2.

16. Ученик выучил в первый день 5 английских слов. В каждый следующий день он выучивал на 2 слова больше, чем в предыдущий. Сколько английских слов выучит ученик в 10-ый день занятий.

Рис. 17 - индивидуальное задание 3 (В-16).

Рис. 18- результат выполнения индивидуального задания 3.

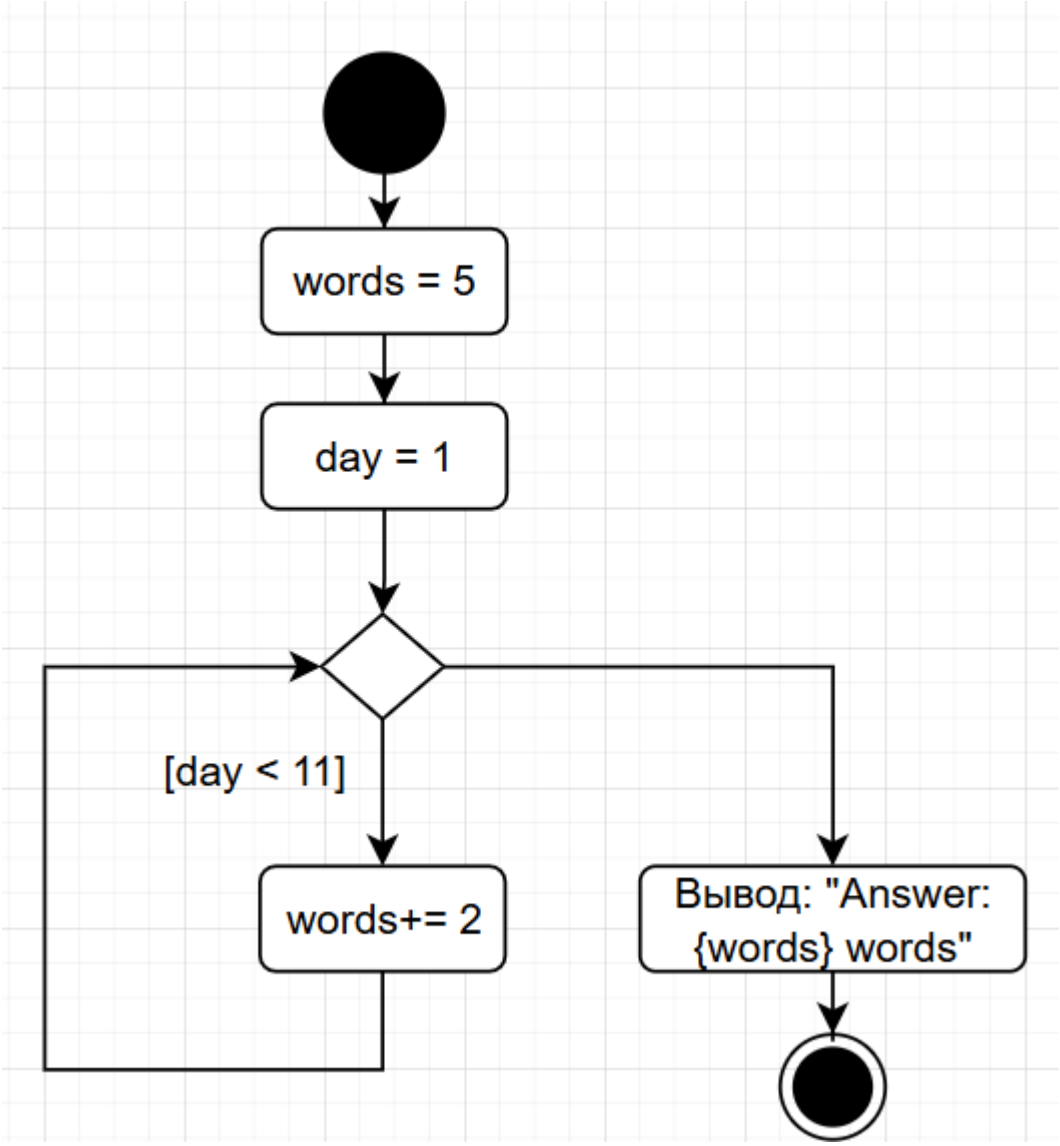

Рис. 19- UML-диаграмма к индивидуальному заданию 3.

7. Функция Бесселя первого рода $I_n(x)$, значение $n = 0, 1, 2, \ldots$ также должно вводиться с клавиатуры

$$I_n(x) = \left(\frac{x}{2}\right)^n \sum_{k=0}^{\infty} \frac{(x^2/4)^k}{k!(k+n)!}.$$

Рис. 20- Задание повышенной сложности. (В-16)

povyshenoy sloznosti.py × Primer_5.py

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import math
import sys

EPS = 1e-10


if __name__ == '__main__':
    x = float(input("Enter x: "))
    n = int(input("Enter n(0,1,2): "))
    if n < 0 or n > 2:
        print("Illegal value of n", file=sys.stderr)
        exit(1)

    a = 1 / (math.factorial(n))
    S, k = a, 0

    while math.fabs(a) > EPS:
        a *= (x * x / 4) / ((k + 1) * (k + n + 1))
        S += a
        k += 1

        factor = (x / 2) ** n
        result = factor * S

        print(f"I_{n}({x}) = {result}")
```

Run povyshenoy sloznosti ×

```
I_1(5.0) = 24.335642142345147
I_1(5.0) = 24.3356421424474
```

Рис. 21- результат выполнения задания повышенной сложности.

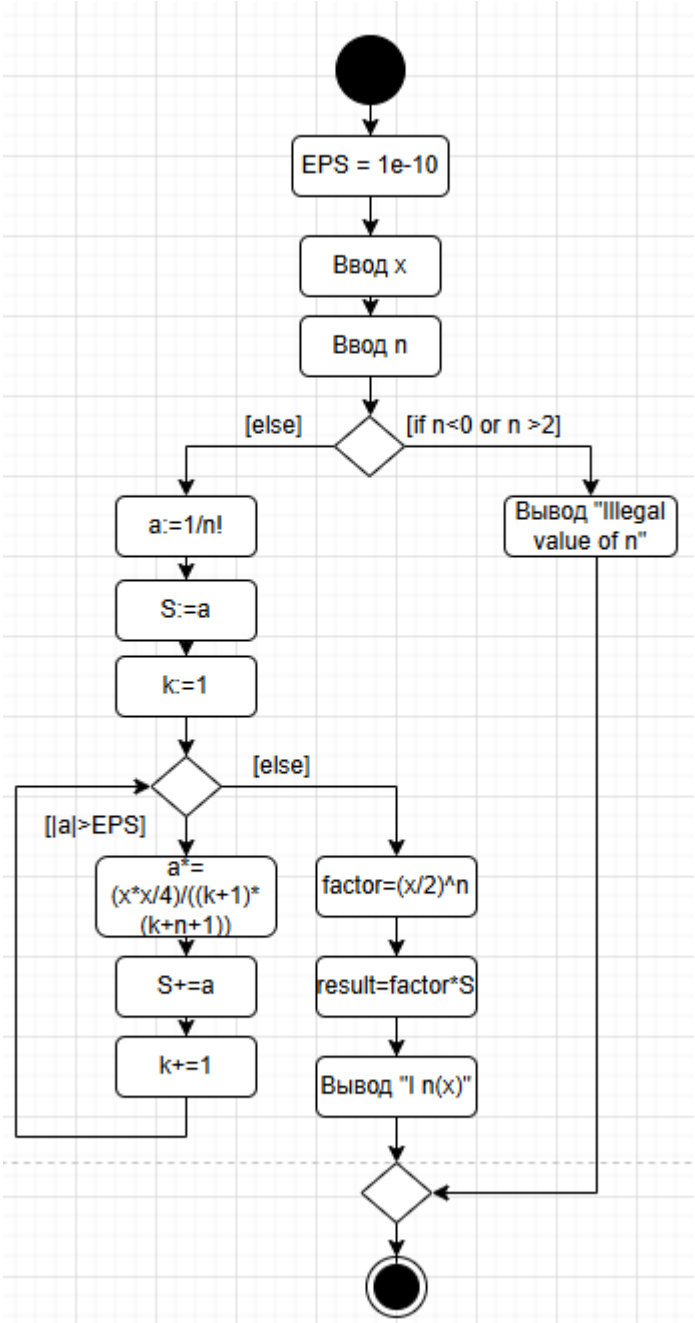

Рис. 22- UML-диаграмма для задания повышенной сложности.

laba_3 Публично

Булавка | Смотреть 0

Главная | 1 Филиал | 0 Теги | Перейти к файлу t | Добавить файл | Код

Тим120903 Задание повышенной сложности 74594a8·сейчас 7 коммитов

.гитиньоре	Обновление .gitignore	5 часов назад
ЛИЦЕНЗИЯ	Первоначальная фиксация	5 дней назад
Primer_2.py	Примеры	1 минуту назад
Primer_3.py	Примеры	1 минуту назад
Primer_4.py	Примеры	1 минуту назад
Primer_5.py	Примеры	1 минуту назад
README.md	Первоначальная фиксация	5 дней назад
индивидуальное задание_3.py	индивидуальные задания	1 минуту назад
индивидуальное задание_1.py	индивидуальные задания	1 минуту назад
индивидуальное задание_2.py	индивидуальные задания	1 минуту назад
повышенной сложности.py	Задание повышенной сложности	сейчас
primer_1.py	Пример_1	2 минуты назад

Рис. 23 - добавленные файлы с коммитами на Github.

Ответы на контрольные вопросы:

1) Диаграммы деятельности. Унифицированный язык моделирования (UML) является стандартным инструментом для создания «чертежей» программного обеспечения. С помощью UML можно визуализировать, специфицировать, конструировать и документировать артефакты программных систем. UML пригоден для моделирования любых систем: от информационных систем масштаба предприятия до распределенных Web-приложений и даже встроенных систем реального времени. Это очень выразительный язык, позволяющий рассмотреть систему со всех точек зрения, имеющих отношение к ее разработке и последующему развертыванию. Несмотря на обилие выразительных возможностей, этот язык прост для понимания и использования.

2) Состояние действия и состояние деятельности. В потоке управления, моделируемом диаграммой деятельности, происходят различные события. Вы можете вычислить выражение, в результате чего изменяется значение некоторого атрибута или возвращается некоторое значение. Также, например, можно выполнить операцию над объектом, послать ему сигнал или даже создать его или уничтожить. Все эти выполняемые атомарные вычисления называются состояниями действия, поскольку каждое из них есть состояние системы, представляющее собой выполнение некоторого действия. Состояния действия изображаются прямоугольниками с закругленными краями. Внутри такого символа можно записывать произвольное выражение. Состояния действия не могут быть подвергнуты декомпозиции. Кроме того, они атомарны. Это значит, что внутри них могут происходить различные события, но выполняемая в состоянии действия работа не может быть прервана. Обычно предполагается, что длительность одного состояния действия занимает неощутимо малое время. В противоположность этому состояния деятельности могут быть подвергнуты дальнейшей декомпозиции, вследствие чего выполняемую деятельность можно представить с помощью других диаграмм

деятельности. Состояния деятельности не являются атомарными, то есть могут быть прерваны. Предполагается, что для их завершения требуется заметное время. Можно считать, что состояние действия - это частный вид состояния деятельности, а конкретнее - такое состояние, которое не может быть подвергнуто дальнейшей декомпозиции. А состояние деятельности можно представлять себе как составное состояние, поток управления которого включает только другие состояния деятельности и действий.

3) Переходы. Когда действие или деятельность в некотором состоянии завершается, поток управления сразу переходит в следующее состояние действия или деятельности. Для описания этого потока используются переходы, показывающие путь из одного состояния действия или деятельности в другое. В UML переход представляется простой линией со стрелкой.

Ветвление. Простые последовательные переходы встречаются наиболее часто, но их одних недостаточно для моделирования любого потока управления. Как и в блок-схеме, вы можете включить в модель ветвление, которое описывает различные пути выполнения в зависимости от значения некоторого булевского выражения. Как видно из рис. 4.3, точка ветвления представляется ромбом. В точку ветвления может входить ровно один переход, а выходить - два или более. Для каждого исходящего перехода задается булевское выражение, которое вычисляется только один раз при входе в точку ветвления. Ни для каких двух исходящих переходов эти сторожевые условия не должны одновременно принимать значение «истина», иначе поток управления окажется неоднозначным. Но эти условия должны покрывать все возможные варианты, иначе поток остановится.

4) Алгоритм разветвляющейся структуры - это алгоритм, в котором вычислительный процесс осуществляется либо по одной, либо по другой ветви, в зависимости от выполнения некоторого условия. Программа разветвляющейся структуры реализует такой алгоритм. В программе разветвляющейся структуры имеется один или несколько условных операторов. Для программной реализации условия используется логическое

выражение. В сложных структурах с большим числом ветвей применяют оператор выбора.

5) Разветвляющиеся алгоритмы отличаются от линейных тем, что в них последовательность выполнения команд зависит от выполнения условия.

6) Условный оператор в программировании — это конструкция, которая проверяет истинность некоторого логического выражения (условия) и в зависимости от результата выполняет тот или иной блок кода. Существуют следующие его формы: конструкция if, конструкция if – else, конструкция if – elif – else

7) В Python используются следующие операторы сравнения: == (равенство), != (неравенство), > (больше), < (меньше), >= (больше или равно), <= (меньше или равно).

8) Простое условие — это одиночное логическое выражение, которое проверяет одно утверждение. Оно возвращает True или False и состоит из одного оператора сравнения или логического оператора.

Пример:

```
x = 5
if x == 5:
    print("x равен 5")
```

9) Составное условие в Python — это конструкция, которая проверяет несколько условий и выполняет только подходящий код. Для этого используются операторы if, elif (сокращение от «else if») и else:

Пример:

```
x = 2
y = 5
if x > 3:
    print(y+x)
elif x < 3:
    print(y-x)
else:
```

print(y*x)

10) При составлении сложных условий широко используются два оператора - так называемые логические И (and) и ИЛИ (or). Чтобы получить True при использовании оператора and, необходимо, чтобы результаты обоих простых выражений, которые связывает данный оператор, были истинными. Если хотя бы в одном случае результатом будет False, то и все сложное выражение будет ложным. Чтобы получить True при использовании оператора or, необходимо, чтобы результат хотя бы одного простого выражения, входящего в состав сложного, был истинным. В случае оператора or сложное выражение становится ложным лишь тогда, когда ложны оба составляющие его простые выражения.

11) Да, оператор ветвления может содержать внутри себя другие ветвления (вложенные ветвления). В этом случае один оператор ветвления можно расположить внутри другого, что позволяет производить выбор более чем из двух вариантов.

12) Алгоритм циклической структуры это тот, который позволяет многократно выполнять одни и те же действия в зависимости от заранее определённых условий.

13) В Python есть два основных типа циклов: for и while. Оператор цикла while выполняет указанный набор инструкций до тех пор, пока условие цикла истинно. Истинность условия определяется также как и в операторе if. Оператор for выполняет указанный набор инструкций заданное количество раз, которое определяется количеством элементов в наборе.

14) Функция range возвращает неизменяемую последовательность чисел в виде объекта range. Она хранит только информацию о значениях start, stop и step и вычисляет значения по мере необходимости. Это значит, что независимо от размера диапазона, который описывает функция range, она всегда будет занимать фиксированный объем памяти.

15) Чтобы организовать перебор значений от 15 до 0 с шагом 2 нужно выполнить следующую программу: list(range(15, -1, -2)

16) Да, циклы могут быть вложенными.

17) Бесконечный цикл образуется, если в условии цикла while не предусмотрено условие выхода. Для выхода из бесконечного цикла в Python можно использовать оператор break.

18) Оператор break предназначен для досрочного прерывания работы цикла while.

19) Оператор continue запускает цикл заново, при этом код, расположенный после данного оператора, не выполняется.

20) В операционной системе по умолчанию присутствуют стандартных потока вывода на консоль: буферизованный поток stdout для вывода данных и информационных сообщений, а также небуферизованный поток stderr для вывода сообщений об ошибках. По умолчанию функция print использует поток stdout. Для того, чтобы использовать поток stderr необходимо передать его в параметре file функции print. Само же определение потоков stdout и stderr находится в стандартном пакете Python sys. Хорошим стилем программирования является наличие вывода ошибок в стандартный поток stderr поскольку вывод в потоки stdout и stderr может обрабатываться как операционной системой, так и сценариями пользователя по-разному.

21) Для того, чтобы использовать поток stderr необходимо передать его в параметре file функции print: print(“Error!”, file=sys.stderr)

22) В Python завершить программу и передать операционной системе заданный код возврата можно посредством функции exit. Вызов exit(1) (0 стоит по умолчанию) приводит к немедленному завершению программы и операционной системе передается 1 в качестве кода возврата, что говорит о том, что в процессе выполнения программы произошли ошибки.

Вывод: в ходе работы были приобретены навыки программирования разветвляющихся алгоритмов и алгоритмов циклической структуры. Были освоены операторы языка Python версии 3.x if, while, for, break и continue, позволяющих реализовывать разветвляющиеся алгоритмы и алгоритмы циклической структуры.